Design: AlphaFold Sequence-to-Structure MCP Server

Thanh Ly · PHYS 7440 GenAI in Physics Research · Assignment 7 · October 2026 · module `mcp_server_ly.py` (server `afdb-sequence-mcp` v1.0.0)

Purpose. For my project I want an LLM agent that can take a coding DNA sequence and answer two questions: what protein structure does it most likely encode, and how far can the AlphaFold prediction be trusted at the residues where my sequence differs from the reference (for example disease variants)? Folding is a free-energy problem, and pLDDT is AlphaFold's per-residue estimate of how well that problem was solved locally. The server gives the agent two read-only tools. No API keys are needed because both data sources are public.

	(a) Endpoint	What it does	(d) External source
1	`find_alphafold_models_for_dna`	Cleans the DNA (FASTA/RNA/IUPAC are accepted), finds the longest ATG ORF in 6 frames, translates it, looks up the protein's MD5 for **identical** AlphaFold models, then annotates and ranks each hit	AlphaFold DB API `GET /api/sequence/summary?id=<md5>&type=md5`; UniProt REST `GET /uniprotkb/accessions`
2	`compare_sequence_to_alphafold`	Fetches a UniProt accession's model and its per-residue pLDDT, builds a confidence profile, aligns an optional query (protein or DNA; BLOSUM62, semi-global) and reports substitutions/indels with local pLDDT and AlphaMissense scores	AlphaFold DB `GET /api/prediction/{acc}`, `/files/AF-…-confidence_v6.json`, `/files/AF-…-aa-substitutions.csv`

(b) inputSchema (as advertised in `tools/list`; descriptions omitted here)

```
{"type":"object","additionalProperties":false,"required":["dna_sequence"],"properties":{
 "dna_sequence":{"type":"string","minLength":30,"maxLength":100000},
 "reading_frame":{"type":"string","enum":["auto","+1","+2","+3","-1","-2","-3"],"default":"auto"},
 "min_protein_length":{"type":"integer","minimum":10,"maximum":5000,"default":30},
 "taxon_id":{"anyOf":[{"type":"integer","minimum":1},{"type":"null"}],"default":null},
 "include_complexes":{"type":"boolean","default":false},
 "max_results":{"type":"integer","minimum":1,"maximum":50,"default":10}}}
```

```
{"type":"object","additionalProperties":false,"required":["uniprot_accession"],"properties":{
 "uniprot_accession":{"type":"string",
   "pattern":"^([OPQ][0-9][A-Z0-9]{3}[0-9]|[A-NR-Z][0-9]([A-Z][A-Z0-9]{2}[0-9]){1,2})(-[0-9]+)?$"},
 "query_sequence":{"anyOf":[{"type":"string","maxLength":100000},{"type":"null"}],"default":null},
 "query_type":{"type":"string","enum":["protein","dna"],"default":"protein"},
 "plddt_threshold":{"type":"number","minimum":0,"maximum":100,"default":70},
 "include_per_residue_plddt":{"type":"boolean","default":false},
 "include_alphamissense":{"type":"boolean","default":true}}}
```

(c) Output format. Both tools publish an `outputSchema` and return `structuredContent` plus the same JSON as text. The envelope is the same for both: `{ok, tool, data, error, warnings[], provenance{sources[], retrieved_at, elapsed_ms, replay_mode}}`.

- Tool 1 `data`: `translation{reading_frame, nt_start, nt_end, protein_sequence, protein_md5, …}`, `alternative_orfs[]`, `match_status` (`exact_match`|`no_exact_match`), `matches[]{model_id, uniprot_accession, organism, taxon_id, reviewed, oligomeric_state, mean_plddt, model_url}`, `best_match`, `next_step`.
- Tool 2 `data`: `model{entry_id, version, organism, urls{pdb,cif,pae_json,…}}`, `confidence{mean/median pLDDT, fraction_very_high/confident/low/very_low, confident_segments[], disordered_segments[], interpretation}`, optional `per_residue_plddt[]`, `comparison{identity, similarity, query/model_coverage, substitutions[]{variant, plddt, plddt_band, alphamissense_pathogenicity, alphamissense_class}, indels[], structure_transfer_confidence, interpretation}`.

Error strategy. Each tool validates its own arguments against the same pydantic models that generate the schema, so a missing or invalid argument gets the same envelope as every other failure. Any failure returns `ok:false`, sets MCP `isError:true`, and fills `error{code, message, retryable, details}`. The codes are `INVALID_INPUT`, `NO_ORF_FOUND`, `NOT_FOUND`, `UPSTREAM_TIMEOUT`, `UPSTREAM_UNAVAILABLE`, `RATE_LIMITED`, `UPSTREAM_BAD_RESPONSE` and `INTERNAL_ERROR`. HTTP calls have a 20 s timeout and are retried twice with exponential backoff on timeouts, 5xx and 429. Sequence errors are caught before any network call. If UniProt is unavailable or AlphaMissense data is missing, the tool still succeeds and adds a `warning`. A 1 h in-memory cache and a polite User-Agent keep the load on the public APIs low.

Transport: stdio, for Claude Desktop, Claude Code and MCP Inspector, or streamable HTTP on localhost.
Reproducibility: `--replay fixtures` serves recorded API responses, so tests and the demo also work offline.